CS307 · VIVA PREPARATION GUIDE

INSIDE, explained simply

Everything you need to know about the paper, our code and our results, without the heavy maths

Paper: Chen et al., “INSIDE: LLMs’ Internal States Retain the Power of Hallucination Detection”, ICLR 2024.
Team: Aayaan Ali Khan (202411001) · Shlok Bhavsar (202411018) · Prapti Chotaliya (202411030)

THE 30-SECOND ANSWER TO “WHAT DID YOU DO?”

Large language models sometimes make things up and sound just as confident when they do. The paper INSIDE detects this by asking the model the same question 10 times and looking *inside* the model at how different those 10 answers are in meaning. If the answers scatter, the model is probably guessing. We took the authors’ code, which did not run, fixed 12 bugs, wrote the missing “feature clipping” part ourselves, made it run on an 8 GB laptop GPU, and reproduced the paper’s main result: their method, EigenScore, is the best of the five detectors, in exactly the paper’s order.

Contents

1 The problem: hallucination
2 Words you must know
3 How others detect it (baselines)
4 Idea 1: EigenScore
5 Idea 2: Feature clipping
6 How detectors are scored
7 What we built
8 The code, file by file
9 Setup and results
10 Limitations
11 End-semester plan
12 Likely viva questions
13 One-page number sheet

1 The problem: hallucination

A **hallucination** is when a language model gives an answer that is fluent and confident but factually wrong.

Example: you ask “In which year did the bridge open?” and the model says “1928” in a perfectly normal tone, when the real answer is 1932. Nothing in the wording tells you it is wrong.

This is dangerous wherever people trust the answer: medical, legal, education, search. So we want a **hallucination detector**: a number that says “this answer is probably wrong, double-check it”.

The hard part

- At the moment the model answers, **we do not know the correct answer**. If we did, we wouldn’t need the model.
- We also don’t want to call Google or a database to fact-check every answer. That is slow and not always possible.
- So the detector must work using **only the model itself**.

The key intuition the whole paper rests on

> ANALOGY
> Ask a student the same question five times. If they really know it, they give the same answer every time. If they are guessing, they say a different thing each time. **Inconsistency is a sign of guessing.** The paper does the same with the model: sample several answers and measure how much they disagree.

Others had this idea before (for example, comparing the words of the answers). INSIDE’s new twist is **where** it measures the disagreement: not in the words, but in the model’s internal “thoughts”. That is why the paper is called INSIDE.

> WHAT INSIDE STANDS FOR
> **IN**ternal **S**tates for halluc**I**nation **DE**tection. It is the name of the overall framework; **EigenScore** is the actual score it computes.

2 Words you must know

Term	Plain meaning
LLM	Large language model, e.g. LLaMA. It writes text one small piece at a time.
Token	One small piece of text (a word or part of a word). The model predicts the next token, then the next, and so on.
Layer	The model is a stack of processing stages. LLaMA-7B has 32. Text goes in at the bottom, the prediction comes out at the top.
Hidden state / embedding	At every layer the model keeps a list of numbers describing "what it currently understands". For LLaMA-7B this list has **4096 numbers**. Think of it as the model's internal notes. Two sentences with the same meaning get similar lists.
Sampling	Generating with some randomness, so the same question can give different answers. This is what lets us ask "10 times".
Temperature (0.5)	How random the sampling is. 0 = always the same answer; higher = more varied. 0.5 is moderate.
Top-p / top-k	Only pick the next token from the most likely options (top 99% of probability / top k tokens). Prevents silly words.
K (= 10)	Number of answers sampled per question.
Logits / probabilities	The model's scores for each possible next token. Older detectors use only these.
Activation / neuron	One of the numbers inside a hidden state. "Extreme activation" = one of these numbers is unusually large.
Penultimate layer	The second-to-last layer (layer 31 of 32).
CoQA	The question-answering dataset we used: a short story followed by questions about it. 7,983 questions.
ROUGE-L	A score (0 to 1) for how much two texts overlap in words, based on the longest common sequence of words.
AUROC	The score used to compare detectors. Explained in Section 6. 0.5 = coin toss, 1.0 = perfect.
4-bit quantisation	Storing each model weight with 4 bits instead of 16, so the model fits in less memory. Slightly less precise.

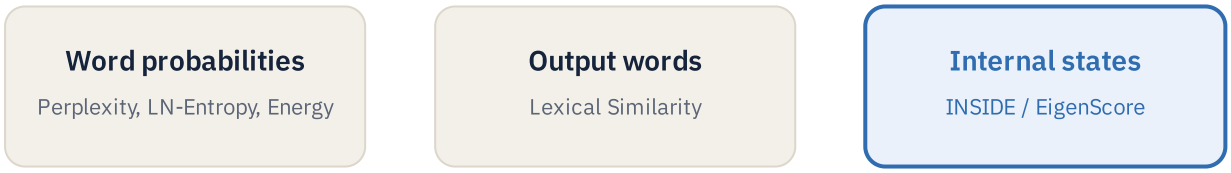
3 How others detect it: the four baselines

The paper compares EigenScore against four existing methods. You should be able to explain each in one line.

Method	What it measures	Weakness
Perplexity	How unsure the model was about each word of *one* answer (average over its tokens).	Being sure of each word ≠ being right. Fluent wrong answers score well.
LN-Entropy (length-normalised entropy)	How unsure the model was, averaged over the K answers, divided by answer length so long answers aren't punished.	Still only looks at word-level confidence, not meaning.
Energy score	A score from the raw logits, originally made to spot inputs that are "out of distribution" (unlike the training data).	Not designed for hallucination. In the paper and in our run it is close to chance (≈55).
Lexical Similarity	Average word overlap (ROUGE-L) between every pair of the K answers. High overlap = consistent.	Compares *words*, not meaning. "1932" and "in the year nineteen thirty-two" look different.

THE GAP INSIDE FILLS

The first three only look at **word probabilities**. The fourth looks at **surface words**. Neither uses the model's rich internal understanding, which is still there at the middle layers. INSIDE uses that.



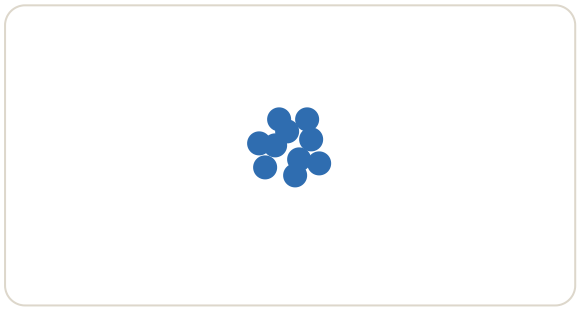
Three places a detector can look. INSIDE is the only one that reads the model's internal notes.

4 Idea 1: EigenScore

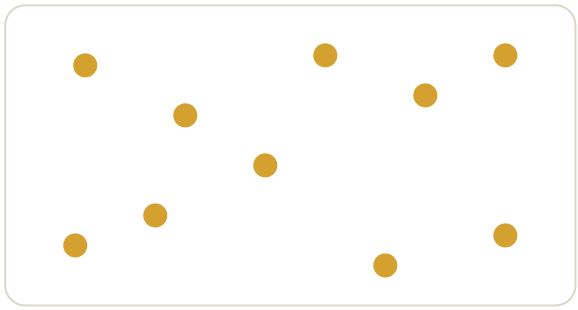
EigenScore answers one question: **“How spread out are the 10 answers, in terms of meaning?”**

Step by step

1. **Ask 10 times.** Sample K = 10 answers to the same question (temperature 0.5).
2. **Take one “meaning fingerprint” per answer.** For each answer, read the hidden state at the **middle layer** (layer 16 of 32) at the **last token** of the answer. That is a list of 4096 numbers that summarises the answer’s meaning.
3. **Measure how differently the 10 fingerprints vary.** Build a small 10 × 10 table (the **covariance matrix**) saying how each pair of answers varies together.
4. **Read off the spread.** Take the table’s **eigenvalues**: each one says how much spread there is in one main direction.
5. **Combine into one number.** EigenScore = the average of the logs of those eigenvalues. **High = answers disagree = likely hallucination.**



Model knows: tight cloud → LOW score



Model guesses: spread cloud → HIGH score

Each dot is one of the 10 answers’ meaning fingerprints (squashed to 2-D for the picture).

The formula, read in plain words

$$\text{EigenScore} = (1/K) \cdot \log\det(\Sigma + \alpha \cdot I) = \text{average of } \log(\text{eigenvalues})$$

- **Σ (covariance):** the 10 × 10 table of how the answers vary together.
- **Eigenvalues:** think of the cloud of points as a blob. Eigenvalues are like its length, width, height… in its main directions. Tight blob = all small; spread blob = large.
- **det (determinant):** the product of the eigenvalues, which is the **volume** of the blob. Log turns the product into a sum, then we average.
- **α = 0.001** is a tiny number added so we never take log(0) when answers are identical (a flat blob has zero volume).

TINY WORKED EXAMPLE

Identical answers → eigenvalues all ≈ 0, plus $\alpha = 0.001$ → $\log_{10}(0.001) = -3$ → score ≈ **−3** (very low, trust it).
Very different answers → eigenvalues like 50, 20, 10… → logs ≈ 1.7, 1.3, 1… → score **much higher** (flag it).
Only the *ranking* of scores matters, so the base of the log (ours uses $\log_{10}$) doesn't change any result.

Why these specific choices?

- **Why the middle layer?** Early layers mostly hold surface information about words; the last layers are busy choosing the exact next word. The middle layers hold the most "meaning". The paper tested every layer (its Figure 3b): the middle ones were clearly best.
- **Why the last token?** The model reads left to right, so the last token has "seen" the whole answer. The paper found it worked better than averaging all tokens.
- **Why only a 10 × 10 table, not 4096 × 4096?** We compare the 10 answers with each other, so the table is answers × answers. It gives the same useful eigenvalues and is tiny and cheap.
- **Why is it called an entropy?** For a bell-shaped cloud of points, log-volume is (up to a constant) its **differential entropy**, a standard measure of uncertainty. So EigenScore is literally "uncertainty about meaning".
- **Why better than Lexical Similarity?** Two answers with different words but the same meaning get close fingerprints, so they aren't wrongly counted as disagreeing.

5 Idea 2: Test-time feature clipping

The blind spot

EigenScore catches guessing. But sometimes the model is **confidently wrong**: it gives the same wrong answer all 10 times. The answers agree, the cloud is tight, the score is low, and the detector wrongly trusts it. The paper calls this a **self-consistent hallucination**.

The paper's fix

The authors noticed that in the **penultimate layer** a few neurons sometimes take **extreme values**, much bigger than usual. They argue these extreme values push the model into being over-confident. So they **cap** them:

- For each of the 4096 neurons, look at its recent values (a **memory bank** of the last **N = 3000 tokens**).
- Find that neuron's normal range: its **0.2nd percentile** (low limit) and **99.8th percentile** (high limit).
- Any value outside the range is pulled back to the limit. Values inside are untouched.
- This is done **while the model is generating**, so it changes the answers themselves. A less over-confident model produces more varied answers, and EigenScore can then notice it.

ANALOGY

A volume limiter on a microphone: normal speech passes unchanged, only sudden shouts are turned down. Here a few "shouting" neurons are turned down so the model stops being overly sure of itself.

$$\mathrm{FC}(h) = \min(\max(h, h_{\min}), h_{\max}) \text{ (clip h to } [h_{\min}, h_{\max}])$$

IMPORTANT FOR THE VIVA

This half of the paper was **not in the authors' released code at all**. We wrote it ourselves from the paper's description (file `func/feature_clip.py`).

In the paper, clipping improves EigenScore on CoQA from **79.3 to 80.4** (AUC_s), only about +1 point. It is a small effect, which matters for our results (Section 9).

6 How detectors are scored

Step 1: label each answer right or wrong

For evaluation only, we do use the reference answer from the dataset. An answer is counted **correct if its ROUGE-L overlap with the reference is above 0.5**. This gives the paper's **AUC_r** (r for ROUGE).

The paper also uses a second rule, **AUC_s**: a sentence-embedding model (nli-roberta-large) says correct if the cosine similarity is above 0.9. We could not use that model, so we report AUC_r only.

Step 2: AUROC, in one sentence

SAY THIS

AUROC is the chance that, if you pick one wrong answer and one right answer at random, the detector gives the wrong one the more suspicious score. 50% is a coin toss, 100% is perfect.

The full name is Area Under the ROC Curve. The ROC curve plots "true positives caught" against "false alarms" as you slide the alarm threshold; the area under it is the AUROC. In the code, EigenScore is **negated** before computing AUROC so that "bigger = more trustworthy" for every method.

Step 3: is a difference real or luck? (paired bootstrap)

With only 250 questions, a 1–3 point difference could be luck. A **bootstrap** checks this:

1. Randomly re-pick 250 questions from our 250, with repeats allowed.
2. Compute both methods' AUROC on that resample and note the difference.
3. Repeat **10,000 times**. The middle 95% of the differences is the **95% confidence interval (CI)**.

"Paired" means both methods are always scored on the same resampled questions, which is fairer. If the CI does not include 0, the difference is unlikely to be luck.

The paper also reports PCC (Pearson correlation between the score and the correctness value). We focus on AUROC.

7 What we built

We started from the authors' official GitHub repository (`alibaba/eigenscore`, commit `ea8062a`). It **could not run**, and it was **missing feature clipping**.

The 12 blocking bugs we fixed (know 4–5 by heart)

#	Problem	Fix
1	Imported a file (`dataeval.TruthfulQA`) that doesn't exist → crashed at start.	Removed the import.
2	Used `np.float`, removed in NumPy 1.24 → every EigenScore function crashed.	Changed to `np.float64`.
3	Paths pointed to the authors' own computer (`/disk1/chenchao/…`).	Paths relative to the repo, changeable by environment variables.
4	Opened the log file before creating its folder.	Create the folder first.
5	Imports only worked when run from inside one folder.	Proper package imports.
6	Loaded a heavy library (selfcheckgpt) for code that was never used.	Load it only where used.
7	Built a BERTScore model from a missing path for a disabled metric.	Removed it.
8	Debugger (`ipdb`) imported at the top of files.	Commented out.
9	Plot saved to a missing folder, then froze waiting for a screen.	Create folder, no pop-up window.
10	File name parsing crashed for normal paths.	Pass the whole path.
11	`.to("cuda")` hard-coded 16 times, ignoring the device option.	Use the tensors' own device.
12	`DEFAULT_DEVICE='cuda:7'` (a GPU we don't have).	Left as is: that file isn't used in our pipeline.

> ONE EXTRA BUG FOUND WHILE RUNNING
>
> To save memory we generate the 10 answers in small batches. The original code computed EigenScore **inside** the batch loop, so only the **last batch (2 of the 10 answers)** was used. We collect all 10 fingerprints first and compute the score once. A test checks this gives the original result when there is only one batch.

Feature clipping, written from scratch

- **Memory bank** = a ring buffer holding the last 3000 tokens' penultimate-layer values (when full, the oldest is overwritten).
- **Thresholds** = per-neuron 0.2% and 99.8% percentiles, recomputed when the bank changes.

- **Warm-up**: no clipping until at least 256 tokens are stored, so limits aren't guessed from a handful of values.
- **Hook**: attached to the penultimate layer with a PyTorch *forward hook* (a function that runs every time that layer produces output), so clipping happens during generation.

Making a 7-billion-parameter model fit an 8 GB laptop GPU

- **4-bit weights (NF4)**: the model shrinks from about 13.5 GB to about 3.9 GB.
- **Smaller batches**: generate the 10 answers 2 at a time (`--num_return_sequences 2`) but still keep K = 10 in total.
- `--low_memory`: normally the model returns every layer's values for every token (gigabytes for long CoQA stories). EigenScore needs only one layer, so a hook saves just the middle layer. A test proves the result is identical.

8 automated tests (all pass)

1. Everything imports.
2. Clipping thresholds equal `numpy.percentile`.
3. Clipping matches the paper's equation.
4. Ring buffer wrap-around and warm-up work.
5. Batched (pooled) EigenScore = original EigenScore.
6. EigenScore is lower for consistent answers than for diverse ones.
7. Low-memory capture = full capture.
8. The code finds the right layers for LLaMA / OPT / Falcon / GPT-2.

8 The code, file by file

File	What it does
`pipeline/generate.py`	Main script. Loads the model and dataset, samples K answers per question, computes all scores, saves them to a `.pkl` file.
`func/metric.py`	The scores: EigenScore (`getEigenIndicator_v0`), perplexity, energy, LN-entropy, lexical similarity.
`func/feature_clip.py`	**Ours.** FeatureClipper (memory bank, thresholds, hook) and the low-memory middle-layer capture.
`func/evalFunc.py`	Labels answers right/wrong (ROUGE-L > 0.5) and computes AUROC for every method.
`func/plot.py`	Draws the ROC curves.
`models/_load_model.py`	Loads the model (local folder, HuggingFace id, or short name), optionally in 4-bit.
`dataeval/coqa.py` etc.	Read the datasets and build prompts.
`tests/test_setup.py`	The 8 tests.
`CHANGES.md`, `RESULTS.md`	List of every fix, and our results with statistics.

What happens for one question

1. Build the prompt: CoQA story + the question.
2. `model.generate(...)` samples answers with temperature 0.5, top-p 0.99, top-k 10, max 256 new tokens, in batches until there are 10. If clipping is on, the hook clips the penultimate layer at every step.
3. From the word probabilities: perplexity, energy, LN-entropy.
4. From the 10 texts: lexical similarity (pairwise ROUGE-L).
5. From the middle-layer, last-token fingerprints: EigenScore.
6. Save everything. Afterwards, `evalFunc` compares each answer to the reference and computes AUROC.

The EigenScore code in 6 lines

```
alpha = 1e-3
selected_layer = int(len(hidden_states[0]) / 2)                  # middle layer
last_embeddings[i] = hidden_states[last_token_i][selected_layer][i]  # one 4096-number
fingerprint per answer
CovMatrix = torch.cov(last_embeddings)                          # 10 x 10 table
u, s, vT = np.linalg.svd(CovMatrix + alpha * np.eye(10))        # s = eigenvalues
eigenScore = np.mean(np.log10(s))                               # average log
```

SVD of a symmetric covariance matrix gives exactly its eigenvalues; that's why the code uses `svd`.

The commands

```
python -m pipeline.generate --model llama2-7b-hf --dataset coqa --load_in_4bit --
low_memory \
   --feature_clip --no_sentsim --num_generations_per_prompt 10 --num_return_sequences 2 \
   --fraction_of_data_to_use 0.03132 --seed 2023
python -m func.evalFunc data/output/llama2-7b-hf_coqa_10/0.pkl --correctness rouge
python -m tests.test_setup        # 8/8 pass
```

9 Setup and results

Our setup vs the paper's

	Paper	Ours
Model	LLaMA-7B, 16-bit	Llama-2-7B, 4-bit
Data	CoQA, 7,983 questions	CoQA, 250 questions
Hardware	not stated	RTX 4060 laptop, 8 GB
K, temperature, top-p	10, 0.5, 0.99	same
top-k	5	10 (repo default)
α, p, N	0.001, 0.2%, 3000	same
Correctness	ROUGE-L and sentence similarity	ROUGE-L only

Because the model, precision and sample size differ, we test whether the **ranking** of methods reproduces, not the exact numbers. The model answered **36.8%** of the 250 questions correctly. Two runs, same seed (2023), same questions: one without and one with clipping. About 6 seconds per question.

Main result: the ranking reproduces exactly

Method (AUROC, CoQA)	Paper	Ours	Ours + clipping
EigenScore	**80.8**	**83.5**	**82.5**
Lexical Similarity	77.8	80.5	79.3
LN-Entropy	73.6	77.5	77.3
Perplexity	68.3	72.3	71.4
Energy	54.7	54.7	53.1

SAY THIS

EigenScore > Lexical Similarity > LN-Entropy > Perplexity > Energy, the same order as the paper. Our numbers are within a few points of the paper's despite a different, compressed model.

A DETAIL EXAMINERS LIKE

The paper's EigenScore of 80.8 **already includes feature clipping** (its baselines don't). So the fair comparison for 80.8 is our clipped run, **82.5**.

Is EigenScore really better than Lexical Similarity?

Paired bootstrap, 10,000 resamples: EigenScore is ahead by **+2.9 points**, 95% CI **[−0.6, +6.5]**, and better in **95%** of resamples. This matches the paper's ~+3 on CoQA. The interval just touches 0, simply because 250 questions is a small sample.

Did feature clipping help?

Change with clipping: **−1.0 point**, 95% CI **[−4.7, +2.6]**, helps in only 30% of resamples. The paper claims **+1.1**. Our interval contains both 0 and +1.1, so the result is **inconclusive**: we can neither confirm nor reject the paper's claim. A 1-point effect needs far more than 250 questions; the paper used 7,983.

We confirmed clipping was really working: **13,749,839** of 1,159,186 × 4096 values were clipped = **0.29%**. That is just under the 0.4% maximum expected (0.2% per side).

We also measured AUC_r where the paper's clipping table uses AUC_s, so they are not exactly the same measurement.

10 Limitations

Of the method itself

- **Cost:** needs 10 to 20 answers per question, so 10 to 20 times the compute of answering once.
- **Needs access to the model's insides:** can't be used with closed APIs like ChatGPT, which only return text.
- **One hand-picked layer:** always the middle layer, yet the paper's own test shows the score changes by several points depending on the layer.
- **Confident mistakes:** still hard; clipping only helps slightly.
- **Short answers:** on TriviaQA (one- or two-word answers) plain perplexity did as well or better in the paper.
- **It only detects** a likely hallucination; it doesn't correct it.

Of our reproduction

- 250 questions → confidence intervals of about ±4 points; a full run would take about 13 hours per setting on our GPU.
- Llama-2 in 4-bit instead of LLaMA in 16-bit.
- ROUGE-L correctness only (no AUC_s).
- top-k 10 instead of 5.

11 End-semester plan

Extension	Idea	Why
Multi-layer EigenScore (main)	**Combine the score from a band of middle layers (e.g. 12–20), with normalised fingerprints, instead of one hand-picked layer.**	**Removes a model-dependent choice; more robust across models.**
Hybrid detector	Combine EigenScore with token confidence (LN-Entropy).	When all answers agree, word-level confidence is the only signal left: targets confident mistakes.
Adaptive K (stretch)	Sample answers one at a time; stop as soon as the score is clearly low or high.	Cuts the 10× cost.
Foundation	Full CoQA (and NQ) run, add sentence-similarity AUROC, use top-k 5; judge every gain with the same paired bootstrap.	

12 Likely viva questions

Q. What is a hallucination and why is it hard to detect?

A. A fluent, confident but wrong answer. It's hard because at answer time there's no correct answer to compare with, and the wording looks normal.

Q. Explain EigenScore in one minute.

A. Ask the model 10 times. For each answer take the middle-layer, last-token hidden state as a meaning fingerprint. Build the 10 × 10 covariance of these fingerprints; its eigenvalues tell how spread out the answers are. The average log eigenvalue is the score: high means the answers disagree, so it's probably a hallucination.

Q. Why internal states instead of the output text?

A. Turning internal states into words throws information away. Hidden states keep the meaning, so two differently worded but equivalent answers look similar, while word overlap would call them different.

Q. What is an eigenvalue here, intuitively?

A. The amount of spread of the answers along one main direction. Small eigenvalues = answers bunched together; large = spread out.

Q. Why add α = 0.001?

A. If the answers are identical the eigenvalues are 0 and log(0) is undefined. α keeps the matrix invertible and sets a floor; it's too small to affect normal cases.

Q. Why the middle layer and the last token?

A. The paper tested layers: middle layers hold the most meaning; early ones are surface-level and the last ones focus on picking the next word. The last token has seen the whole answer, and beat averaging all tokens.

Q. How is EigenScore related to entropy?

A. For a Gaussian (bell-shaped) cloud, the log-determinant of the covariance is its differential entropy up to a constant. So EigenScore measures uncertainty in meaning-space.

Q. What is a self-consistent hallucination and how does clipping help?

A. When the model gives the same wrong answer every time, so consistency checks trust it. Clipping caps rare extreme neuron values in the penultimate layer, which the paper links to over-confidence, so the model varies more and EigenScore can catch it.

Q. How did you implement clipping?

A. A FeatureClipper class: ring-buffer memory of the last 3000 tokens, per-neuron 0.2% / 99.8% percentiles, clamp values outside, 256-token warm-up, attached as a forward hook on the penultimate layer so it acts during generation. Tested against numpy.percentile.

Q. What is AUROC?

A. The probability that a randomly chosen wrong answer gets a more suspicious score than a randomly chosen right one. 0.5 = chance, 1 = perfect.

Q. How did you decide if an answer was correct?

A. ROUGE-L overlap with the reference answer above 0.5 (the paper's AUC_r). Used only for evaluation, never by the detector.

Q. Your numbers are higher than the paper's. Why?

A. Different model (Llama-2 vs LLaMA-1), 4-bit precision, only 250 questions and top-k 10. That's why we compare the ranking, which matched exactly. With ±4-point intervals, a few points of difference is expected.

Q. Why only 250 questions?

A. An 8 GB laptop GPU at about 6 seconds per question: the full 7,983 would take about 13 hours per setting. Planned for end-sem.

Q. Feature clipping made things worse in your run. Is the paper wrong?

A. No conclusion either way. The change was −1.0 with a 95% interval of [−4.7, +2.6], which includes both 0 and the paper's +1.1. The effect is too small to measure with 250 questions. We did verify clipping was active (0.29% of values clipped).

Q. What is a bootstrap confidence interval?

A. Resample our 250 questions with replacement 10,000 times, recompute the difference each time; the middle 95% of those differences is the interval. If it excludes 0, the difference is unlikely to be luck.

Q. Why is Energy so bad?

A. It was built to detect unusual inputs (out-of-distribution), not wrong answers. The paper includes it to show that OOD methods don't transfer; it's near chance both there and in our run.

Q. Why 4-bit? Doesn't it change the model?

A. Slightly, but it was the only way to fit a 7B model plus its hidden states into 8 GB (13.5 GB → 3.9 GB). Since the ranking still matched the paper, the method seems robust to it.

Q. What did `--low_memory` do and how do you know it's correct?

A. Instead of storing every layer for every token, a hook saves only the middle layer EigenScore needs. A test shows the scores are identical to the full method.

Q. What was wrong with the original code?

A. It couldn't start (missing module, removed NumPy type, hard-coded paths to the authors' machine and GPU, etc.: 12 blocking bugs), feature clipping was missing, and when batching it used only 2 of the 10 answers. All listed in CHANGES.md.

Q. Why is the covariance 10 × 10 and not 4096 × 4096?

A. We compare answers with each other, so it's answers × answers. It contains the same non-zero spread information and is far cheaper.

Q. What is K and does it matter?

A. Number of sampled answers. The paper found accuracy rises until about 15 and then flattens; K = 20 was its best trade-off. We used 10 like its main experiments.

Q. Can this be used on ChatGPT?

A. Not directly: it needs the model's hidden states, which closed APIs don't expose. It works for open models like LLaMA.

Q. What will you add by end-sem and why?

A. Multi-layer EigenScore (remove the hand-picked layer), a hybrid with token confidence (catch confident mistakes), and adaptive K (reduce cost), all evaluated at full scale with the same bootstrap.

Q. What is temperature and why 0.5?

A. It controls randomness in sampling. We need some variety to compare answers, but not so much that even known facts come out differently. 0.5 is the paper's setting.

Q. Who did what?

A. [Fill in before the viva: e.g. bug fixes, clipping implementation, experiments and statistics, report and video.]

---

13 One-page number sheet

Method settings	
K (answers per question)	**10**
Temperature / top-p / top-k	0.5 / 0.99 / 10 (paper 5)
Layer used	middle: 16 of 32
Token used	last token of the answer
Fingerprint size	4096 numbers
α	0.001
Clipping layer	penultimate (31 of 32)
Clipping percentile p	0.2% each side
Memory bank N	3000 tokens
Warm-up before clipping	256 tokens
Correct if	ROUGE-L > 0.5

Our experiment	
Model	Llama-2-7B, 4-bit NF4
GPU	RTX 4060 laptop, 8 GB
Questions	250 CoQA (paper 7,983)
Seed	2023
Model accuracy	36.8%
Speed	~6 s / question
Bugs fixed	12 (+1 batching bug)
Tests	8 / 8 pass
Memory: 16-bit → 4-bit	13.5 GB → 3.9 GB
Values clipped	0.29% (max expected 0.4%)

AUROC (CoQA)	Paper	Ours	Ours + FC
EigenScore	**80.8 (with FC)**	**83.5**	**82.5**
Lexical Similarity	77.8	80.5	79.3
LN-Entropy	73.6	77.5	77.3
Perplexity	68.3	72.3	71.4
Energy	54.7	54.7	53.1

Statistical checks (paired bootstrap, 10,000 resamples)	Difference	95% CI	Verdict
EigenScore − Lexical Similarity	+2.9	[−0.6, +6.5]	better in 95% of resamples; matches paper's ~+3
Feature clipping effect on EigenScore	−1.0	[−4.7, +2.6]	inconclusive (paper: +1.1)

Paper facts	
Venue	ICLR 2024, Chen et al. (Alibaba); code: github.com/alibaba/eigenscore
Models in paper	LLaMA-7B, LLaMA-13B, OPT-6.7B
Datasets in paper	CoQA, SQuAD, NQ, TriviaQA
Clipping gain in paper	CoQA EigenScore 79.3 → 80.4 (AUC_s)
Best K in paper	gains flatten after ~15; K = 20 suggested